

SECURITY AUDIT REPORT

Kula EIP Suite

CLASSIFICATION

Public Report

VERSION

1.0

DATE

July 2, 2026

PREPARED FOR

Kula GroupVerichains Lab
info@verichains.io
www.verichains.io

Abbreviations

NAME	DESCRIPTION
Smart contract	A computer protocol intended to digitally facilitate, verify or enforce the negotiation or performance of a contract.
Ethereum	An open source platform based on blockchain technology to create and distribute smart contracts and decentralized applications.
Ether (ETH)	A cryptocurrency whose blockchain is generated by the Ethereum platform. Ether is used for payment of transactions and computing services in the Ethereum network.
Solidity	A contract-oriented, high-level language for implementing smart contracts for the Ethereum platform.
Solc	A compiler for Solidity.

Executive Summary

This Security Audit Report was prepared by Verichains on July 2, 2026. We would like to thank the Kula Group for trusting Verichains in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the Kula EIP Suite. The scope of the audit is limited to the source code files provided to Verichains. Verichains completed the assessment using manual, static, and dynamic analysis techniques.

- **During the audit process, the audit team identified several vulnerabilities.**

Table of Contents

01	Management Summary	06
1.1	About Kula EIP Suite	06
1.2	About Verichains	06
1.3	Audit Methodology	07
1.4	Disclaimer	08
1.5	Acceptance Minute	08
02	Audit Overview	09
2.1	Audit Scope	09
2.2	Overview	11
2.3	Asset-Bound Token Registry	12
2.4	Canonical Document Bundle Anchor	12
2.5	Directional Transfer Domain Registry	12
2.6	Compliance Event Log	12
2.7	Impact Snapshot Log	12
2.8	NAV Snapshot Oracle	13
03	Findings 20 ISSUES	14
3.1	MED Single provider's mismatched <code>navBasis</code> permanently bricks <code>aggregatedNAV</code>	FIXED 14
3.2	MED Revoked NAV provider's snapshot is permanently non-evictable	FIXED 15
3.3	MED Permanent token-binding-key squatting	FIXED 16
3.4	LOW Token registry agreement check bypassed by non-standard return data	FIXED 17
3.5	LOW BundleAnchorVerifier constructor skips ERC-165 check when the call reverts	FIXED 19
3.6	LOW Revoked registrar retains bind/re-attest power over its own anchors	FIXED 20
3.7	LOW Zero <code>gracePeriod</code> produces a non-cancellable graceful revocation	FIXED 21
3.8	LOW <code>latestEventByType</code> returns recording order, not occurrence order	FIXED 22
3.9	LOW <code>reattest()</code> permits reducing the expiry	FIXED 23
3.10	LOW <code>supersedeMethodology</code> allows scheduling supersession for an ordinal arbitrarily far in the future	FIXED 24
3.11	LOW Permanent <code>(subjectId, role)</code> bundle-slot squatting	FIXED 25
3.12	INFO Redundant <code> revocation.finalized</code> in <code>finalizeRevocation</code>	FIXED 26
3.13	INFO <code>isActive()</code> and <code>isBound()</code> return false for nonexistent anchors	FIXED 27
3.14	INFO Zero <code>role</code> / <code>subjectId</code> accepted in <code>DocumentBundleAnchor</code>	FIXED 28
3.15	INFO No zero-value validation for domain and asset-class identifiers	FIXED 29
3.16	INFO Immediate-revocation evidence is not stored on-chain	FIXED 30
3.17	INFO <code>AnchorMetadataLib.validate()</code> does not enforce <code>attestationDate < expiresAt</code>	FIXED 31
3.18	INFO <code>correctedByIndex</code> sentinel is undocumented and inconsistent with <code>correctsIndex</code>	FIXED 32
3.19	INFO Correction-of-correction chaining is unspecified (EIP-4 and EIP-6)	FIXED 33
3.20	INFO <code>BundleHashLib.sortEntries()</code> uses an $O(n^2)$ bubble sort	FIXED 34

04 Version History

36

SECTION 01

Management Summary

1.1 About Kula EIP Suite

Kula is a decentralised impact investment firm revolutionizing traditional finance through the tokenization of real-world assets and community-driven governance. Rather than prioritizing extractive returns, Kula’s model emphasizes participation and stewardship, enabling token holders to influence decisions on managing assets including land, water, timber, agriculture, and commodities. Operating through Kula VASP Limited in Mauritius under Financial Services Commission regulation, the firm maintains compliance and credibility in the financial ecosystem.

Unlike conventional investment structures that distribute dividends, Kula reinvests capital to grow its ecosystem. Token holders benefit through governance rights, performance of tokenized assets, and transaction-based revenue streams. The firm’s core mission focuses on unlocking value in under-served regions while promoting inclusive, sustainable prosperity that benefits communities for the long term.

1.2 About Verichains

Since 2017, Verichains has been a pioneer and leading blockchain security firm in APAC, with extensive expertise in security, cryptography, and core blockchain technology. Over 200 clients trust us to protect assets valued at more than \$50 billion, including several high-profile clients such as BNB Chain, Bybit, Aptos, Sui, Polygon, Klaytn, Wemix, Line Corp, Ronin Network, and Kyber Network.

With in-depth research and development of blockchain technology, Verichains offers a range of security services, including blockchain protocol and smart contract security audit, mobile application protection, key management solutions, on-chain risk monitoring, and red team/penetration testing services.

Verichains is a certified CREST penetration testing member, delivering professional testing services at the highest global standards.

Homepage	https://www.verichains.io
Blog	https://blog.verichains.io
Email	info@verichains.io
Twitter	https://twitter.com/Verichains
LinkedIn	https://www.linkedin.com/company/verichains

1.3 Audit Methodology

Our security audit process for smart contracts includes two steps:

01

Smart contract code is scanned and tested for commonly known and more specific vulnerabilities using public tools and **RK87**, our in-house smart contract security analysis tool.

02

Manual audit of the code for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that were considered during the audit of the smart contract:

COMMONLY KNOWN VULNERABILITIES

- Integer Overflow and Underflow
- Timestamp Dependence
- Race Conditions
- Transaction-Ordering Dependence
- DoS with (Unexpected) revert
- DoS with Block Gas Limit
- Gas Usage, Gas Limit and Loops
- Redundant fallback function
- Unsafe type Inference
- Reentrancy
- Explicit visibility of functions and state variables
- Logic Flaws

For vulnerabilities, we categorize the findings into categories as listed in the table below, depending on their severity level:

SEVERITY LEVEL	DESCRIPTION
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

Table 1 · Severity levels

1.4 Disclaimer

Kula Group acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. Kula Group understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, Kula Group agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.5 Acceptance Minute

This final report served by Verichains to Kula Group will be considered an Acceptance Minute. Within 7 days, if no further responses or reports are received from Kula Group, the final report will be considered fully accepted by Kula Group without the signature.

SECTION 02

Audit Overview

2.1 Audit Scope

This audit focused on identifying security flaws in the code and the design of the Kula EIP Suite. It was conducted on the commit below from the git repository <https://github.com/KulaDao/titled-asset-standards>.

Commit `caa9b05b1aee786317790c3d2d9c6a67f6e52f7f`

ASSET REGISTRY

FILE	SHA-256
<code>erc-asset-registry/src/libraries/AssetRegistryConstants.sol</code>	<code>2e353ccfedfbe1a88c481b1296cfbca3fd48a827469b0935a03882fc54f5d264</code>
<code>erc-asset-registry/src/reference/AssetAnchorRegistry.sol</code>	<code>30110db8e68e3412754ce70b2ded02bcb6f091e45b65b544e89b4b1c1aebd362</code>
<code>erc-asset-registry/src/interfaces/IAssetBoundToken.sol</code>	<code>4ff798ccf15de64fe06bb9c1481c97bf7dce6e19d6d857eefdae81d7d66f9116</code>
<code>erc-asset-registry/src/libraries/AnchorMetadataLib.sol</code>	<code>7c99d8619f8bffe086a489ea56b7b5be7c5368e798c88308035e3ba5888778d8</code>
<code>erc-asset-registry/src/interfaces/IAssetAnchorRegistry.sol</code>	<code>f7395c8c3aa8b80ecf62c55dbe3fce9390d41089875a83432e2c6f164db8bd3b</code>

DOCUMENT BUNDLE

FILE	SHA-256
<code>erc-document-bundle-anchor/src/libraries/BundleHashLib.sol</code>	<code>4a967906c11da0ee6fb355c2ef19ee391b39ef30883098c36912ff19bd7e20ba</code>
<code>erc-document-bundle-anchor/src/interfaces/IDocumentBundleAnchor.sol</code>	<code>7683b1216b5b17ea97bfc8a329d20ff66c6985ba9699180ce1170a59cf54aac2</code>

`erc-document-bundle-
anchor/src/reference/
BundleAnchorVerifier.sol`

`cd8856a1d7c93f172461ec0690f3318c833c0a06b9942b88
bed8e129ee6ead63`

`erc-document-bundle-
anchor/src/reference/
DocumentBundleAnchor.sol`

`f30f1206b08eb0b5b705d603dec2099ed66506d781d3d5ab
b81d82e44b18a1fc`

TRANSFER DOMAIN

FILE	SHA-256
<code>erc-transfer-domain/src/ libraries/TransferRoute Lib.sol</code>	<code>120175ce59e8d0e9b90a7af785182e50eb910d46b06a1f70 d8e41cb6a9e89357</code>
<code>erc-transfer-domain/src/ reference/TransferDomain Registry.sol</code>	<code>42dce6739e7cfee41d5d294ca7f0dacc85f665f4e754625f 4bde7eb19ca8db9d</code>
<code>erc-transfer-domain/src/ interfaces/ITransfer DomainRegistry.sol</code>	<code>7799a08a805a140f8be8380648124d823ed36291c1c79518 ed7c5d3fbf6c18aa</code>
<code>erc-transfer-domain/src/ interfaces/IGracefulRoute Revocation.sol</code>	<code>79db76ec3eb2e451b417c00dd01ab72caa44a6daed1c6f52 e7b14fd0db80ac42</code>
<code>erc-transfer-domain/ src/reference/Graceful TransferDomainRegistry. sol</code>	<code>8f2d88c09e332033bcba8e16eb456599c719a4edf8ed0b93 da36d321696731b6</code>

COMPLIANCE EVENT

FILE	SHA-256
<code>erc-compliance-event- log/src/interfaces/ IComplianceEventLog.sol</code>	<code>5a3ab0b21450773e8ea40a9f0d223c7dd7d523693be08ecc 3d665c8f3bb26cdb</code>
<code>erc-compliance-event-log/ src/reference/Compliance EventLog.sol</code>	<code>6de46fd0349709b2f82b92f5dccfcf27869eb0f9951182d8 9b2ada775617b239</code>
<code>erc-compliance-event-log/ src/libraries/Compliance Constants.sol</code>	<code>72e29ebd4742e543f6355b4992308bd5fdd4a6326e443cc5 081ff1d8dda55a0e</code>

IMPACT SNAPSHOT

FILE	SHA-256
<code>erc-impact-snapshot/src/interfaces/IImpactSnapshotLog.sol</code>	0c1f94520c0cb7a19ba0c7873a32635557bdc8b492e3edfedd308089ea7e0fc6
<code>erc-impact-snapshot/src/libraries/ImpactConstants.sol</code>	3f6c0d7a23c8a4596236feeb43120b4698ab33172fa93d97256361a164c5cf08
<code>erc-impact-snapshot/src/interfaces/IMethodologyVersioning.sol</code>	706ad3700b996d0fa271c668608acc7cbe631429aa9c4f6e9ba8d90177ddd51f
<code>erc-impact-snapshot/src/interfaces/IImpactAttestation.sol</code>	b16cee558d30c3f08c374da54c07a9a03d39d059335b518f541e41ad419770c8
<code>erc-impact-snapshot/src/reference/ImpactSnapshotLog.sol</code>	b1b006d51a69286e3f5f1e978a734e0f90b1a97875fd0844142a488e1974d9fc

NAV ORACLE

FILE	SHA-256
<code>erc-nav-oracle/src/interfaces/INAVSnapshotOracle.sol</code>	8dd5e229d3bd81cafc52d65cd96dd26b266ee42d3ecfe3a42cb7cc30b23b0732
<code>erc-nav-oracle/src/reference/NAVSnapshotOracle.sol</code>	92795379f6e44914e941b2d58c155a405eebbf8a548206557f73c971af306896
<code>erc-nav-oracle/src/libraries/NAVConstants.sol</code>	cd19f4c6f712ec7d2dfd0483e569183c888138d20066b340859056ef356747e6
<code>erc-nav-oracle/src/interfaces/INAVAggregation.sol</code>	e8f01d84e16b6325ffe124e1c3e12914f2c4329c02564c98efec12edf57c5a4d

2.2 Overview

The contracts comprises six composable EIP specifications that provide on-chain infrastructure for tokenized real-world assets. Each EIP addresses a narrow, well-defined problem and can be adopted independently. Together they form a complete infrastructure layer for compliant, auditable, and interoperable tokenized assets.

Asset-Bound Token Registry

Asset-Bound Token Registry establishes the canonical on-chain identity for a real-world asset by registering a dual-hash anchor - a legal document commitment (`legalHash`) and an evidence commitment (`evidenceHash`). The resulting `anchorId` is computed deterministically as `keccak256(abi.encode(legalHash, evidenceHash))` and serves as the universal subject identifier across all companion EIPs. The anchor can then be permanently bound to a token contract (whole-contract binding for ERC-20) or to a specific token ID (per-token binding for ERC-721/1155). The registry enforces a strict lifecycle: anchors are active upon registration, can be re-attested to extend their expiry, and can be permanently deactivated by an admin. Access is governed by `REGISTRAR_ROLE` for registration and binding, and `DEFAULT_ADMIN_ROLE` for deactivation and administrative overrides.

Canonical Document Bundle Anchor

Canonical Document Bundle Anchor anchors a deterministic, order-independent bundle hash derived from a set of document entries to a `(subjectId, role)` namespace on-chain. The bundle hash is computed off-chain using `BundleHashLib`, which applies a total order over all leaf fields before hashing so that any permutation of the same document set produces the same hash. Each `(subjectId, role)` slot has exactly one active bundle at any time. When a document set is updated, `supersedeBundle()` marks the old record as superseded and installs the new bundle as active. Superseded records remain permanently queryable, preserving full document history. Only the original anchorer or an admin may supersede a bundle.

Directional Transfer Domain Registry

Directional Transfer Domain Registry is a token-agnostic registry for answering whether a transfer route from `sourceDomain` to `destinationDomain` is permitted for a given `assetClass`. Domains and asset classes are opaque `bytes32` identifiers whose semantics are defined by the consuming application. Routes are directional - `A → B` is independent of `B → A` - and every set or revoke operation requires a non-zero evidence hash. The suite includes a graceful revocation extension that introduces a fixed, immutable grace period before a revocation takes effect. During the grace period, the revocation can be cancelled; after it expires, the revocation is lazily enforced by `isRoutePermitted()` returning `false` without requiring an explicit finalization transaction.

Compliance Event Log

Compliance Event Log provides an append-only on-chain log of structured compliance events - KYC checks, sanctions screening, freeze actions, and audit events - bound to a `(subjectId, eventType)` namespace. Each event carries an evidence hash, a structured payload (up to 2048 bytes), up to 10 involved parties, and temporal data with a maximum backdate window of 30 days. Events cannot be modified or deleted; corrections are recorded as new events linked to their target via a fork-free correction chain, ensuring each event can be corrected at most once. The `currentEventIndex()` function resolves any event to its terminal (uncorrected) successor by walking the chain.

Impact Snapshot Log

Impact Snapshot Log provides an append-only on-chain log for structured, auditable impact data - carbon offsets, energy generation, water treatment, and other ESG indicators - bound to a `(subjectId, indicatorId)` namespace. Snapshots are period-slotted: each `(subjectId, indicatorId, periodStart, periodEnd)` combination can have only one original snapshot, with subsequent data recorded as corrections via a fork-free chain. Independent attestors can endorse or reject snapshots without modifying them, and self-attestation by the reporter is prohibited. Methodology versioning is strictly

future-only: **effectiveFromOrdinal** must be greater than or equal to the current indicator snapshot count, preventing retroactive methodology changes that would break the chain of custody for previously attested data.

NAV Snapshot Oracle

NAV Snapshot Oracle is a permissioned, multi-provider NAV oracle that provides subject-keyed, provider-attributed Net Asset Value snapshots keyed by (**subjectId**, **currency**). Multiple providers publish NAV data for the same stream, and the system computes a median-based aggregated NAV once a configurable quorum of eligible providers is met. Decimal values are normalized to the highest submitted precision before aggregation, and corrected snapshots are excluded from the aggregation set. The oracle enforces dual staleness thresholds - a **heartbeat** for publication freshness and a **maxValuationAge** for valuation freshness - and emits deviation alerts when the spread across provider submissions exceeds a configured basis-point threshold. The NAV basis (per-unit, per-share, or total) is locked per stream at configuration time and cannot be changed once set.

SECTION 03

Findings

#	TITLE	SEVERITY	STATUS
1	Single provider's mismatched <code>navBasis</code> permanently bricks <code>aggregatedNAV</code>	MEDIUM	Fixed
2	Revoked NAV provider's snapshot is permanently non-evictable	MEDIUM	Fixed
3	Permanent token-binding-key squatting	MEDIUM	Fixed
4	Token registry agreement check bypassed by non-standard return data	LOW	Fixed
5	BundleAnchorVerifier constructor skips ERC-165 check when the call reverts	LOW	Fixed
6	Revoked registrar retains bind/re-attest power over its own anchors	LOW	Fixed
7	Zero <code>gracePeriod</code> produces a non-cancellable graceful revocation	LOW	Fixed
8	<code>latestEventByType</code> returns recording order, not occurrence order	LOW	Fixed
9	<code>reattest()</code> permits reducing the expiry	LOW	Fixed
10	<code>supersedeMethodology</code> allows scheduling supersession for an ordinal arbitrarily far in the future	LOW	Fixed
11	Permanent <code>(subjectId, role)</code> bundle-slot squatting	LOW	Fixed
12	Redundant <code> revocation.finalized</code> in <code>finalizeRevocation</code>	INFO	Fixed
13	<code>isActive()</code> and <code>isBound()</code> return false for nonexistent anchors	INFO	Fixed
14	Zero <code>role</code> / <code>subjectId</code> accepted in <code>DocumentBundleAnchor</code>	INFO	Fixed
15	No zero-value validation for domain and asset-class identifiers	INFO	Fixed
16	Immediate-revocation evidence is not stored on-chain	INFO	Fixed
17	<code>AnchorMetadataLib.validate()</code> does not enforce <code>attestationDate < expiresAt</code>	INFO	Fixed
18	<code>correctedByIndex</code> sentinel is undocumented and inconsistent with <code>correctsIndex</code>	INFO	Fixed

19	Correction-of-correction chaining is unspecified (EIP-4 and EIP-6)	INFO	Fixed
20	<code>BundleHashLib.sortEntries()</code> uses an $O(n^2)$ bubble sort	INFO	Fixed

MEDIUM

• FIXED

M-01

Single provider's mismatched `navBasis` permanently bricks `aggregatedNAV`

POSITION

`eip-6-nav-oracle/src/reference/NAVSnapshotOracle.sol` - lines 545-595 `_aggregatedSet`

`eip-6-nav-oracle/src/reference/NAVSnapshotOracle.sol` - lines 497-505 `_updateLatestEligibleTimestamp`

DESCRIPTION

`publishNAV` lets each provider submit any `navBasis` for a given valuation timestamp, but `aggregatedNAV` requires every eligible provider at the latest eligible timestamp to share one basis and reverts otherwise. Because the latest eligible timestamp only ever advances, one provider submitting a wrong basis at that timestamp makes `aggregatedNAV/providerSubmission` revert for everyone, and the offending provider cannot fix it (a correction must keep the same `navBasis`, line 131) - the feed stays down until a new, later timestamp independently reaches quorum with a consistent basis, a DoS of the oracle's core aggregation that downstream consumers (e.g. a lending protocol pricing collateral) depend on.

solidity

```
// _aggregatedSet
if (used == 0) {
    commonBasis = snap.navBasis;
} else {
    require(snap.navBasis == commonBasis, "NAVSnapshotOracle: mixed navBasis"); //
    reverts whole call
}
```

RECOMMENDATION

Instead of allowing providers to submit the `navBasis` dynamically, the `navBasis` should be explicitly defined and initialized by an `admin`.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-nav-oracle`.
- **June 25, 2026:** The team has read and accepted the recommendation.

MEDIUM

• FIXED

M-02

Revoked NAV provider's snapshot is permanently non-evictable

POSITION

`packages/erc-nav-oracle/src/reference/NAVSnapshotOracle.sol` - lines 132-134 [publishNAV](#)

DESCRIPTION

NAV corrections are the only way to neutralize a published snapshot. However, the correction path strictly requires the caller to be the original provider (`target.provider == msg.sender`), with no `DEFAULT_ADMIN_ROLE` override or evict function. If a malicious provider's role is revoked, their poisoned snapshots become permanently uncorrectable, remaining in the median calculation indefinitely.

```
solidity

NAVSnapshot storage target = _snapshots[streamKey][params.correctsIndex];
require(target.correctedByIndex == NO_CORRECTED_BY, "NAVSnapshotOracle: target
already corrected");
require(target.provider == msg.sender, "NAVSnapshotOracle: provider mismatch"); //
No admin bypass
```

RECOMMENDATION

Add a `DEFAULT_ADMIN_ROLE` override to the NAV correction path, or add an admin-only `invalidateSnapshot(streamKey, index)` function.

UPDATE

- **July 1, 2026:** The team has added the `invalidateSnapshot` for only admin permission..

MEDIUM

• FIXED

M-03

Permanent token-binding-key squatting

POSITION

packages/erc-asset-registry/src/reference/AssetAnchorRegistry.sol - lines 83-86 [_bind](#)

DESCRIPTION

The `_bind` function permanently assigns a `bindingKey` to an `anchorId`. It requires the key to be empty but lacks an admin bypass or an `unbind` function. A rogue registrar can bind their anchor to any token's slot, permanently squatting it. Even the admin cannot overwrite this binding or bind the correct anchor afterward.

 solidity

```
bytes32 bindingKey = _tokenBindingKey(token, bindingScope, tokenId);  
// No admin bypass for occupied slots  
require(_boundAnchorByTokenBinding[bindingKey] == bytes32(0), "AssetAnchorRegistry:  
token binding already bound");  
...  
_boundAnchorByTokenBinding[bindingKey] = anchorId; // Never cleared
```

RECOMMENDATION

Add an admin-only `clearBinding(bindingKey)` or allow admins to bypass the emptiness check to overwrite an existing binding.

UPDATE

- **July 1, 2026:** The team has added `invalidateTokenBinding` function for admin.

LOW

• FIXED

L-01

Token registry agreement check bypassed by non-standard return data

POSITION

eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol - lines 113-119
requireTokenRegistryAgreement

DESCRIPTION

The agreement guard only enforces the registry match when the `anchorRegistry()` staticcall succeeds and returns exactly 32 bytes; a token whose call reverts, returns empty data, or returns a non-32-byte payload silently skips the check. A token that declares a different registry through a non-standard encoding can therefore be bound here, defeating the “if a token names its registry it must match” invariant (mitigated by binding being access-controlled, hence Low).

```
solidity

(bool ok, bytes memory data)
= token.staticcall(abi.encodeWithSignature("anchorRegistry()"));
if (ok && data.length == 32) { // exact 32 → non-standard
    returns skip the check
    address declared = abi.decode(data, (address));
    require(declared == address(this), "AssetAnchorRegistry: token registry
mismatch");
}
```

RECOMMENDATION

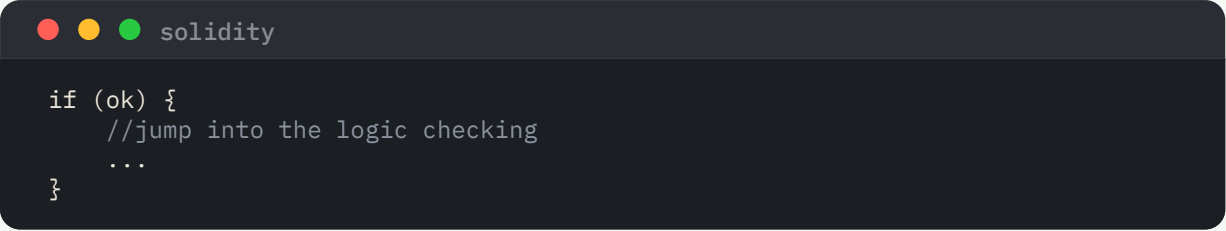
Use an inclusive `>= 32` length check so ABI-encoded address returns with trailing data are still validated, and treat a successful-but-unparseable response as a mismatch rather than silently allowing it:

```
solidity

if (ok && data.length >= 32) {
    address declared = abi.decode(data, (address));
    require(declared == address(this), "AssetAnchorRegistry: token registry
mismatch");
}
```

UPDATE

- **July 1, 2026:** The team has read and accepted the recommendation.
- **June 30, 2026:** The package has been changed to `erc-asset-registry`. The checking mechanism would be wrong if `ok == true` and `data == null`. The `data.length > 0` check should be removed from the `if` statement.



```
solidity
```

```
if (ok) {  
    //jump into the logic checking  
    ...  
}
```

LOW

• FIXED

L-02

BundleAnchorVerifier constructor skips ERC-165 check when the call reverts

POSITION

eip-2-document-bundle/src/reference/BundleAnchorVerifier.sol - lines 36-41 (constructor)

DESCRIPTION

If the registry's `supportsInterface()` reverts, `ok` is false and the `require` is skipped entirely, so any contract can be wired in as the registry without supporting `IDocumentBundleAnchor`; additionally the exact `data.length == 32` rejects otherwise-valid bool returns carrying trailing bytes. Impact is limited to deploy-time misconfiguration since the deployer controls the registry address.

 solidity

```
if (ok) {
    require(data.length == 32 && abi.decode(data, (bool)), "BundleAnchorVerifier:
    unsupported registry");
} // no else branch: a reverting supportsInterface() passes validation
```

RECOMMENDATION

Treat a reverted ERC-165 call as a failure (revert in the `else` branch) and relax the length check to `>= 32`:

 solidity

```
if (ok) {
    require(data.length >= 32 && abi.decode(data, (bool)),
    "BundleAnchorVerifier: unsupported registry");
} else {
    revert("BundleAnchorVerifier: registry ERC-165 check failed");
}
```

UPDATE

- **June 30, 2026:** The package has been changed to `erc-document-bundle-anchor`.
- **June 25, 2026:** The team has read and accepted the recommendation.

LOW

• FIXED

L-03

Revoked registrar retains bind/re-attest power over its own anchors

POSITION

eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol - lines 38-41 `bindToken`

eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol - lines 169-172 `reattest`

DESCRIPTION

Authorization for `bindToken()` and `reattest()` is based on the immutable creator mapping `_registeredBy[anchorId] == msg.sender`, not on currently holding `REGISTRAR_ROLE`. After an admin revokes `REGISTRAR_ROLE` from a compromised operator, that operator can still bind tokens to and extend the expiry of every anchor it previously created, forcing the admin to manually `deactivateAnchor()` each one to contain the damage.

```
solidity

require(
  _registeredBy[anchorId] == msg.sender || hasRole(DEFAULT_ADMIN_ROLE,
    msg.sender),
  "AssetAnchorRegistry: not authorized to bind"
);
```

RECOMMENDATION

Require the caller to still hold an active `REGISTRAR_ROLE` in addition to being the original registrar (admins exempt):

```
solidity

require(
  (hasRole(REGISTRAR_ROLE, msg.sender) && _registeredBy[anchorId] ==
    msg.sender) ||
  hasRole(DEFAULT_ADMIN_ROLE, msg.sender),
  "AssetAnchorRegistry: not authorized"
);
```

If the creator-ownership model is intentional, document that role revocation does not revoke anchor control.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-asset-registry`.
- **June 25, 2026:** The team has read and accepted the recommendation.

LOW

• FIXED

L-04

Zero `gracePeriod` produces a non-cancellable graceful revocation

POSITION

`eip-3-transfer-domain/src/reference/GracefulTransferDomainRegistry.sol` - line 13 (constructor)

`eip-3-transfer-domain/src/reference/GracefulTransferDomainRegistry.sol` - lines 62, 89
`initiateRevocation/cancelRevocation`

DESCRIPTION

The constructor accepts `gracePeriod_ == 0` without validation. With a zero grace period, `initiateRevocation` sets `effectiveAt = block.timestamp`, so `cancelRevocation` always reverts “grace period expired” - the cancel window is zero - while the revocation stays `pending`, blocking further `initiateRevocation` until someone finalizes. The contract silently degrades into an immediate-revocation registry that needs an extra finalize step.

```
solidity

constructor(address admin, uint64 gracePeriod_) TransferDomainRegistry(admin) {
    gracePeriod = gracePeriod_;           // no gracePeriod_ > 0 check
}
// effectiveAt = block.timestamp + 0;  cancel: require(block.timestamp <
effectiveAt) // never true
```

RECOMMENDATION

Require `gracePeriod_ > 0` in the constructor:

```
solidity

require(gracePeriod_ > 0, "GracefulTransferDomainRegistry: zero grace
period");
```

If zero is intentionally supported, document and test the full lifecycle.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-transfer-domain`.
- **June 25, 2026:** The team has read and accepted the recommendation.

LOW

• FIXED

L-05

latestEventByType returns recording order, not occurrence order

POSITION

eip-4-compliance-event/src/reference/ComplianceEventLog.sol - lines 138-142

DESCRIPTION

The function returns the most recently recorded event index for a type, but the 30-day backdate window (`MAX_BACKDATE_SECONDS`) lets a recorder append a backdated event after a more recent one, making the older-occurring event the “latest”. A consumer using this for current KYC/compliance state can silently read stale status.

solidity

```
uint256[] storage indices = _eventsByType[subjectId][eventType];
require(indices.length != 0, "ComplianceEventLog: no events for type");
return indices[indices.length - 1]; // last *recorded*, not max occurredAt
```

RECOMMENDATION

Document in the interface that this returns recording-order, not occurrence-order; consumers needing occurrence recency must iterate `eventByTypeAt` and compare `occurredAt`.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-compliance-event-log`. The function name changed into `lastRecordedEventByType`.
- **June 25, 2026:** Since the behavior has now been confirmed, we suggest renaming the function to avoid confusion.

LOW

• FIXED

L-06

`reattest()` permits reducing the expiry

POSITION

`eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol` - line 173

DESCRIPTION

The only constraint on `newExpiresAt` is that it be in the future; nothing requires it to be $\geq$ the current expiry. A registrar can shorten the validity window (e.g. to one second out), effectively invalidating an anchor without emitting `AnchorDeactivated` or setting `active = false`, blurring the line with the permanent-deactivation path.

 solidity

```
require(newExpiresAt > block.timestamp, "AssetAnchorRegistry: expiresAt must be future");  
// missing: newExpiresAt >= current _metadata[anchorId].expiresAt
```

RECOMMENDATION

If shortening is unintended, require `newExpiresAt` to be $\geq$ the existing expiry:

 solidity

```
require(newExpiresAt >= _metadata[anchorId].expiresAt, "AssetAnchorRegistry:  
new expiry before current");
```

Otherwise document expiry reduction as a deliberate administrative capability distinct from deactivation.

UPDATE

- **July 1, 2026:** Both monotonic guards are now present in `reattest()`: `require(newExpiresAt >= meta.expiresAt, ...)` and `require(newAttestationDate >= meta.attestationDate, ...)`.
- **June 30, 2026:** The package has been changed to `erc-asset-registry`. Also, the `reattest` function prevents reducing the expiration time. But there is a similar mechanism on `newAttestationDate`. It only validates that the value is nonzero (`!= 0`), not in the future (`<= block.timestamp`), and earlier than `newExpiresAt` (`< newExpiresAt`). As a result, a registrar can arbitrarily move the attestation date backward, causing a newly re-attested anchor to appear as though it was attested much earlier than it actually was. To preserve the chronological integrity of attestations, the function should enforce `newAttestationDate >= meta.attestationDate`.
- **June 25, 2026:** The team has read and accepted the recommendation for `newExpiresAt`.

LOW

• FIXED

L-07

`supersedeMethodology` allows scheduling supersession for an ordinal arbitrarily far in the future

POSITION

`eip-5-impact-snapshot/src/reference/ImpactSnapshotLog.sol` - lines 245-282 `supersedeMethodology()`

DESCRIPTION

In `ImpactSnapshotLog`, the `supersedeMethodology()` function allows a reporter to schedule a new methodology to take effect at a specific future ordinal (`effectiveFromOrdinal`). The function requires that `effectiveFromOrdinal >= currentOrdinal`, but lacks an upper bound restriction on how far into the future it can be scheduled.

Since the contract only allows one pending methodology supersession at a time for a given subject and indicator (`require(!_pendingMethodology[subjectId][indicatorId].pending)`), a reporter could schedule a supersession for an ordinal that is arbitrarily far in the future (e.g., `type(uint256).max`). This effectively locks the pending slot, that prevents any other legitimate methodology updates from being scheduled until that distant ordinal is reached.

RECOMMENDATION

Consider adding an upper bound check for `effectiveFromOrdinal`.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-impact-snapshot`. And the team has added a require statement:

```
require(
    effectiveFromOrdinal <= currentOrdinal + MAX_METHODODOLOGY_LOOKAHEAD,
    "ImpactSnapshotLog: effectiveFromOrdinal too far in the future"
);
```

LOW

• FIXED

L-08

Permanent (subjectId, role) bundle-slot squatting

POSITION

packages/erc-document-bundle-anchor/src/reference/DocumentBundleAnchor.sol - lines 40-71 `anchorBundle`/`supersedeBundle`

DESCRIPTION

The `anchorBundle` function allows any `ANCHOR_ROLE` holder to occupy an empty (subjectId, role) slot. Once occupied, legitimate operators cannot `anchorBundle` (slot occupied) nor `supersedeBundle` (requires `old.anchoredBy == msg.sender`). While the admin can recover the slot via `supersedeBundle`, the non-admin legitimate operator is permanently locked out.

```
solidity

// In anchorBundle:
require(_activeSlots[slotKey] == bytes32(0), "DocumentBundleAnchor: active slot
occupied, use supersedeBundle");

// In supersedeBundle:
require(
    old.anchoredBy == msg.sender || hasRole(DEFAULT_ADMIN_ROLE, msg.sender),
    "DocumentBundleAnchor: not authorized to supersede"
);
```

RECOMMENDATION

Bind slot authority to a principal (e.g., record the slot owner), or explicitly restrict `anchorBundle` for a given `subjectId` to operators authorized for that subject.

UPDATE

- **July 2, 2026:** The team has acknowledged and implemented an update for this issue.
- **July 1, 2026:** Attacker with no relation to the subject squats the slot; legit operator blocked from both `anchorBundle` and `supersedeBundle`. Admin recovery via `supersedeBundle` is itself front-runnable by the still-`ANCHOR_ROLE`-holding squatter if the role isn't revoked first.

INFO

• FIXED

I-01

Redundant `|| revocation.finalized` in `finalizeRevocation`

POSITION

`eip-3-transfer-domain/src/reference/GracefulTransferDomainRegistry.sol` - lines 100-101

DESCRIPTION

`pending` and `finalized` are mutually exclusive (finalize sets `pending=false`, `finalized=true` atomically), so the `|| revocation.finalized` branch in the first `require` only changes the error message for an already-finalized route - the immediately-following `!finalized` check already covers that case. No behavioral impact; code-clarity only.

 solidity

```
require(revocation.pending || revocation.finalized, "GracefulTransferDomainRegistry: no revocation");
require(!revocation.finalized, "GracefulTransferDomainRegistry: already finalized");
```

RECOMMENDATION

Drop `|| revocation.finalized` from the first condition:

 solidity

```
require(revocation.pending, "GracefulTransferDomainRegistry: no revocation");
```

UPDATE

- **June 30, 2026:** The package has been changed to `erc-transfer-domain`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-02

`isActive()` and `isBound()` return false for nonexistent anchors

POSITION

`eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol` - lines 200-204 `isActive`

`eip-1-asset-registry/src/reference/AssetAnchorRegistry.sol` - lines 69-71 `isBound`

DESCRIPTION

Both functions return `false` for an unknown `anchorId`, conflating “wrong/typo id”, “deactivated”, and “expired”, whereas `getAnchor/getMetadata/registeredBy` revert on nonexistent anchors. A caller can mistake a wrong id or wrong registry for a legitimately inactive anchor.

```
solidity

function isBound(bytes32 anchorId) external view returns (bool) {
    return _records[anchorId].boundToken != address(0); // false for nonexistent
    too
}
```

RECOMMENDATION

Add an existence guard to both functions:

```
solidity

require(_records[anchorId].registeredAt != 0, "AssetAnchorRegistry: anchor not
found");
```

Or document the non-reverting semantics explicitly.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-asset-registry`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-03

Zero `role` / `subjectId` accepted in `DocumentBundleAnchor`

POSITION

`eip-2-document-bundle/src/reference/DocumentBundleAnchor.sol` - lines 24-41 `anchorBundle`

`eip-2-document-bundle/src/reference/DocumentBundleAnchor.sol` - lines 45-78
`supersedeBundle`

DESCRIPTION

`bundleHash` and `documentCount` are validated, but `subjectId` and `role` are not; anchoring with `bytes32(0)` keys creates a zero-namespace slot indistinguishable from an unset one, so a consumer querying `activeBundle` with an accidental zero key receives a result instead of an error. No fund/authorization impact (anchoring is role-gated).

```
solidity

function anchorBundle(
    bytes32 bundleHash,
    bytes32 subjectId,    // no zero-check
    bytes32 role,        // no zero-check
    ...
)
```

RECOMMENDATION

Reject zero `subjectId`/`role` in both functions, or explicitly document `bytes32(0)` as a permitted default namespace.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-document-bundle-anchor`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-04

No zero-value validation for domain and asset-class identifiers

POSITION

eip-3-transfer-domain/src/reference/TransferDomainRegistry.sol - lines 38-53 [setRoute](#)

DESCRIPTION

`setRoute()/revokeRoute()` reject a zero evidence hash but accept `bytes32(0)` for `sourceDomain`, `destinationDomain`, or `assetClass`, so a misconfigured call silently creates a “null” route key that is indistinguishable from an intentional zero-namespace route.

 solidity

```
require(permissionEvidenceHash != bytes32(0), "TransferDomainRegistry: zero
permissionEvidenceHash");
// sourceDomain, destinationDomain, assetClass unchecked
```

RECOMMENDATION

Add zero checks for the three route-key components, or document zero identifiers as intentional wildcards.

 solidity

```
require(sourceDomain != bytes32(0), "TransferDomainRegistry: zero
sourceDomain");
require(destinationDomain != bytes32(0), "TransferDomainRegistry: zero
destinationDomain");
require(assetClass != bytes32(0), "TransferDomainRegistry: zero assetClass");
```

UPDATE

- **June 30, 2026:** The package has been changed to `erc-transfer-domain`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-05

Immediate-revocation evidence is not stored on-chain

POSITION

eip-3-transfer-domain/src/reference/TransferDomainRegistry.sol - lines 55-70 `revokeRoute`

eip-3-transfer-domain/src/interfaces/ITransferDomainRegistry.sol - `Route` struct

DESCRIPTION

`Route` has no field for revocation evidence, so `revokeRoute()` only emits `revocationEvidenceHash` in an event while `GracefulTransferDomainRegistry` stores it in `Revocation`. Graceful-revocation proofs are directly queryable but immediate ones must be reconstructed from logs - an asymmetry, not a vulnerability.

```
solidity

_routes[key].permitted = false;
_routes[key].effectiveAt = effectiveAt;
emit RouteRevoked(..., revocationEvidenceHash, effectiveAt); // hash never stored
```

RECOMMENDATION

Add a `revocationEvidenceHash` field to `Route` and persist it, or document the event-only design choice in the interface.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-transfer-domain`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-06

AnchorMetadataLib.validate() does not enforce attestationDate < expiresAt

POSITION

eip-1-asset-registry/src/libraries/AnchorMetadataLib.sol - lines 23-29 `validate`

DESCRIPTION

The library only checks non-zero/non-empty fields; the chronological constraint `attestationDate < expiresAt` is enforced only in `AssetAnchorRegistry._register()` (line 135). Any future or external reuse of `validate()` without the registry's extra checks could accept temporally invalid metadata. No impact on the current reference contract.

```
solidity

require(m.attestationDate != 0, "AnchorMetadataLib: missing attestationDate");
require(m.expiresAt != 0, "AnchorMetadataLib: missing expiresAt");
// missing: m.attestationDate < m.expiresAt
```

RECOMMENDATION

Move the chronological invariant into the library's `validate()` so it travels with the metadata type:

```
solidity

require(m.attestationDate < m.expiresAt, "AnchorMetadataLib: expiresAt not
after attestationDate");
```

UPDATE

- **June 30, 2026:** The package has been changed to `erc-asset-registry`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-07

`correctedByIndex` sentinel is undocumented and inconsistent with `correctsIndex`

POSITION

`eip-4-compliance-event/src/interfaces/IComplianceEventLog.sol` - line 4, `ComplianceEvent` struct

`eip-4-compliance-event/src/reference/ComplianceEventLog.sol` - lines 79, 102

DESCRIPTION

`correctsIndex` uses the exported `NO_CORRECTION = type(uint256).max` sentinel while `correctedByIndex` uses an undocumented `0` for “not yet corrected”. The `0` sentinel is safe in practice (a correction event can never live at index 0), but the inconsistency invites a consumer to wrongly test `correctedByIndex == NO_CORRECTION`, which would treat every superseded event as still current. Code is internally correct; this is a documentation/ergonomics issue.

 solidity

```
uint256 correctsIndex;    // sentinel NO_CORRECTION (type(uint256).max) - named
uint256 correctedByIndex; // sentinel 0 - unnamed, undocumented
```

RECOMMENDATION

Export a named companion constant for the `0` sentinel and add NatSpec to both fields explaining why the two sentinels differ and why `0` is safe.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-compliance-event-log`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-08

Correction-of-correction chaining is unspecified (EIP-4 and EIP-6)

POSITION

`eip-4-compliance-event/src/reference/ComplianceEventLog.sol` - lines 78-84

`eip-6-nav-oracle/src/reference/NAVSnapshotOracle.sol` - lines 127-136

DESCRIPTION

Both contracts only check `target.correctedByIndex == 0` (“not already corrected”) and never check `target.correctsIndex == NO_CORRECTION`, so a correction event/snapshot can itself be corrected, forming chains. The READMEs call corrections “fork-free” but are silent on chaining. EIP-6 aggregation handles chains correctly (it skips any `correctedByIndex != 0`), so this is a spec/documentation gap rather than a correctness bug.

```
require(target.correctedByIndex == 0, "...: target already corrected");  
// no check that target.correctsIndex == NO_CORRECTION → chains allowed  
target.correctedByIndex = eventIndex; // (snapshotIndex in EIP-6)
```

RECOMMENDATION

Either forbid correcting a correction:

```
require(target.correctsIndex == NO_CORRECTION, "cannot correct a correction");
```

Or document chaining explicitly and provide an on-chain terminal-walk helper as EIP-5 does.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-compliance-event-log` and `erc-nav-oracle`.
- **June 25, 2026:** The team has read and accepted the recommendation.

INFO

• FIXED

I-09

BundleHashLib.sortEntries() uses an $O(n^2)$ bubble sort

POSITION

eip-2-document-bundle/src/libraries/BundleHashLib.sol - lines 59-71 `sortEntries`, lines 39-55 `_computeBundleHashSorted`

DESCRIPTION

`sortEntries` is an $O(n^2)$ bubble sort with per-swap struct copies, and `_computeBundleHashSorted` re-allocates via repeated `abi.encodePacked`. The reference `DocumentBundleAnchor` does not call these on-chain (it takes a precomputed hash), so the cost only materializes if a downstream contract computes bundle hashes on-chain for large bundles.

```
solidity
for (uint256 i = 0; i < n; i++) {
    for (uint256 j = 0; j + 1 < n - i; j++) {
        if (_gt(entries[j], entries[j + 1])) { /* swap full struct */ }
    }
}
```

RECOMMENDATION

Document that canonical bundle hashing should be done off-chain with pre-sorted entries passed in; if on-chain sorting is required for large sets, use a more efficient algorithm.

UPDATE

- **June 30, 2026:** The package has been changed to `erc-document-bundle-anchor`.
- **June 25, 2026:** The team has read and accepted the recommendation.

SECTION 04

Version History

VERSION	DATE	STATUS / CHANGE	CREATED BY
1.0	July 2, 2026	Public Report	Verichains

Table 2 · Report versions history



Verichains Lab · info@verichains.io · www.verichains.io